

CICLO DE COMPLEMENTACIÓN

LICENCIATURA EN CIENCIA DE DATOS

CLCD 182 - SEMINARIO DE
ACTUALIZACIÓN I

Clases

Semana 02

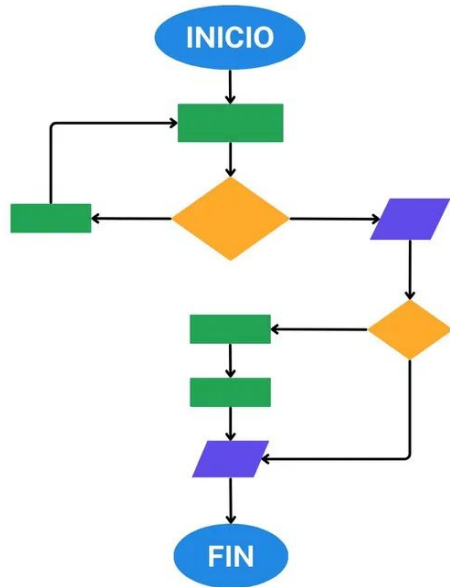




Temas a Desarrollar

- Control de flujo
- ¿Qué es el Control de Flujo?
- Sentencia if
- Sentencia for
- Función range
- Sentencia while
- Función match
- Sentencia break
- Sentencia continue
- Función zip
- Función enumerate
- List comprehensions

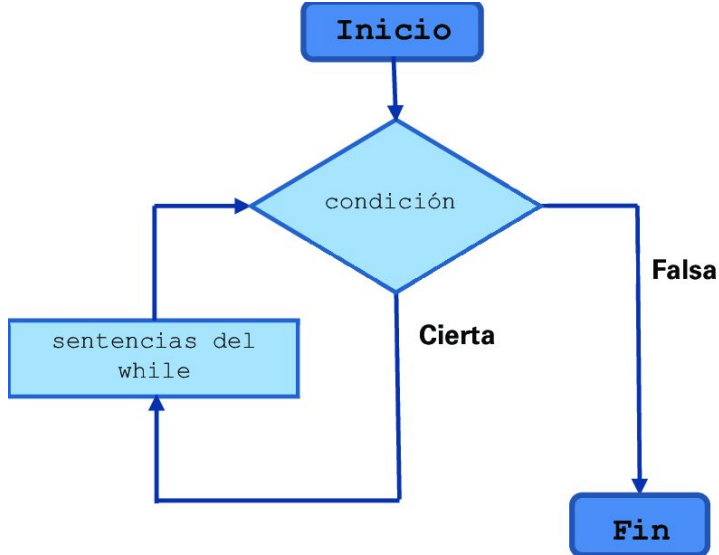
Control de flujo



Objetivos:

- Entender los conceptos fundamentales de control de flujo.
- Aprender a utilizar las estructuras de control más comunes en Python.
- Escribir código más eficiente y legible.

¿Qué es el Control de Flujo?



- Definición: El control de flujo determina el orden en que se ejecutan las instrucciones de un programa.
- Importancia: Permite tomar decisiones, repetir acciones y crear programas más complejos.



Sentencia if

Sintaxis:

```
if condición:
    # Bloque de código si la condición es verdadera
else:
    # Bloque de código si la condición es falsa
```

Ejemplos:

- Verificar si un número es par o impar.
- Tomar decisiones basadas en el valor de una variable.
- Condiciones compuestas: and, or, not



Sentencia for

Ejecución incondicional: Se ejecuta un número conocido de veces. Itera sobre secuencias como pueden ser las listas, tuplas, cadenas de caracteres, etc.

Sintaxis:

```
for elemento in secuencia:  
    # Código a ejecutar para cada elemento
```

Ejemplo:

- Iterar sobre una lista de nombres y saludar a cada uno.
- Buscar una palabra en una lista de renglones.
- Procesar las filas de un excel.



Función range

Crea una secuencia inmutable de números enteros.

Se utiliza principalmente con bucles **for**.

Declaración:

```
range(start, stop, step)
```

Sintaxis:

```
for i in range(2, 11, 2):  
    print(i)
```

Ejemplos:

- `range(5)`: Genera números del 0 al 4 (5 no incluido).
- `range(2, 7)`: Genera números del 2 al 6 (7 no incluido).
- `range(1, 10, 2)`: Genera números impares del 1 al 9.



Sentencia while

Ejecución condicional: Se ejecuta mientras una condición sea verdadera.

Sintaxis:

```
while condición:  
    # Código a ejecutar mientras la condición sea verdadera
```

Ejemplo:

- Un contador que se incrementa hasta alcanzar un valor determinado.

Importante: El riesgo de este bucle es iterar infinitamente si la condición nunca se vuelve falsa.



Sentencia break

Sale del bucle más cercano.

Sintaxis:

```
break
```

Ejemplo:

- Buscar un elemento específico en una lista y detener la búsqueda si se encuentra.
- Saltar elementos que no cumplan una determinada condición.



Sentencia continue

Salta a la siguiente iteración del bucle más cercano ignorando las sentencias que se encuentra debajo de él.

Sintaxis:

```
continue
```

Ejemplo:

- Saltar elementos que no cumplan una determinada condición.



Sentencia match

En Python contamos **match**.

Ejemplo:

```
match x:
    case 10:
        print("It's 10")
    case 20:
        print("It's 20")
    case _:
        print("It's neither 10 nor 20")
```

Alternativas:

- if-elif-else: Para múltiples condiciones.
- Diccionarios: Para mapear valores a funciones.



Función zip

Combina elementos de múltiples iterables para recorrerlos todos al mismo tiempo. Siempre itera como máximo la de menor longitud.

Declaración:

```
zip(iterable_1, iterable_2, ..., iterable_n)
```

Sintaxis:

```
for valor_1, valor_2, ..., valor_n in zip(iter_1, iter_2, ..., iter_n):  
    print(valor_1, valor_2, ..., valor_n)
```

Ejemplo:

- Iterar sobre dos listas simultáneamente.



Función enumerate

Obtiene el índice y el valor de cada elemento en una secuencia.

Declaración:

```
enumerate(iterable_1)
```

Sintaxis:

```
for posicion, valor in enumerate(iterable_1):  
    print(posicion, ' - ', valor)
```

Ejemplo:

- Obtener el índice de cada elemento en una lista.



List Comprehensions

Sintaxis concisa para crear secuencias de forma elegante.

Sintaxis:

```
lista_concisa = [i**2 for i in range(1,11)]
```

Ejemplo:

- Crear una lista de los cuadrados de los números del 1 al 10.
- Filtrar elementos.
- Crear listas de listas.



Iteradores e Iterables

Los iteradores se utilizan para recorrer secuencias, por ejemplo list, set, tuple, dict, etc.

Los iterables son objetos que los puedo recorrer con iteradores.

Ejemplos:

```
lista = [5, 4, 9, 2]
for elemento in lista:
    print(elemento, end=', ')
```

Ejemplos:

```
iterador = iter(lista)
print(f'Posición 0: {next(iterador)}') # Posición 0: 5
print(f'Posición 1: {next(iterador)}') # Posición 1: 4
print(f'Posición 2: {next(iterador)}') # Posición 2: 9
print(f'Posición 3: {next(iterador)}') # Posición 3: 2
```